M12 Review / Repair - Teaching RC2

OPEN ONLY AFTER SAVED ATTEMPT

P01 repair - Why Spark exists: driver, executors, partitions and tasks

Large datasets or expensive transformations need parallel work across cores/machines while a coordinator plans the job.

Repair principle: The **driver** runs your application logic/plans jobs and coordinates executors.

Key proof: task -> stage work on partition

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P02 repair - PySpark setup, SparkSession and laptop-safe local mode

PySpark requires JVM/runtime integration; environment errors should be diagnosed, not solved by reinstalling random versions.

Repair principle: Reuse the supplied project requirements; install pyspark in a module venv.

Key proof: runtime -> no unexplained Java gateway error

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P03 repair - DataFrames and explicit schemas

Schema inference can be convenient but unstable/expensive for production sources; wrong types propagate into distributed transformations.

Repair principle: DataFrame is a distributed table-like dataset with named typed columns.

Key proof: qty operation -> numeric expression works

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P04 repair - Spark SQL and expression equivalence

Teams use both styles. The correctness/performance question is about the logical/physical plan, not API tribalism.

Repair principle: Register a temp view to query a DataFrame with Spark SQL.

Key proof: optimizer -> plans both forms

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P05 repair - Lazy transformations and actions

Spark builds a plan first so it can optimize multiple transformations before touching the full dataset.

Repair principle: Transformations (select/filter/withColumn/join/groupBy) generally create new lazy DataFrames/plans.

Key proof: count/write/collect -> action

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P06 repair - Jobs, stages, tasks and physical plans

Distributed performance debugging starts with the plan: where data is exchanged, how many partitions/tasks, and which operators dominate.

Repair principle: An action creates one or more jobs.

Key proof: PartitionFilters -> partition pruning clues

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P07 repair - repartition vs coalesce

Partition count affects parallelism, output file count and later shuffle cost.

Repair principle: repartition(n, cols...) can increase/decrease partitions and generally shuffles to redistribute.

Key proof: one tiny demo file -> coalesce(1), not general pattern

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P08 repair - Aggregations and shuffle cost

groupBy over distributed data must bring equal keys together, which often creates heavy network/disk shuffle.

Repair principle: GroupBy/aggregate by non-partitioned key usually needs shuffle.

Key proof: total sales -> reconciles to source completed total

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P09 repair - Joins and broadcast strategy

A normal distributed join may shuffle both sides by key. Broadcasting a small dimension can avoid moving the large side.

Repair principle: Spark can choose broadcast hash join when one side is below/advised threshold and join type supports it.

Key proof: left join check -> row count/unmatched keys

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P10 repair - Join correctness, fan-out and reconciliation

Distributed joins can multiply rows exactly like SQL/pandas joins. Spark scale makes the mistake expensive.

Repair principle: State left and right grain/uniqueness before join.

Key proof: measure total -> no hidden multiplication

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P11 repair - Data skew

If one key owns a huge share of rows, shuffle sends much of the data to one partition/task, limiting parallelism.

Repair principle: Skew means uneven data/work distribution across partitions/tasks, often driven by hot keys.

Key proof: remedy -> must preserve correctness

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P12 repair - Adaptive Query Execution (AQE)

Static estimates can be wrong. Spark can adapt join/shuffle choices after seeing actual intermediate data.

Repair principle: Adaptive Query Execution can coalesce shuffle partitions and adjust some join/skew strategies at runtime when enabled.

Key proof: runtime join choice -> driver collect explosion

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.